



**Government College Of Engineering Aurangabad
Chhatrapati Sambhajanagar**

Report on

Object Detection using YOLO-World

SubmittedBy

BT24S05F001 – Vikas Gholap

BT24S05F003 – Rutik Ghose

In partial fulfillment for the
CSHNC0020 : Advance Computer Vision

Under The Guidance Of

Dr. V.J.Pawar

Government College of Engineering Aurangabad ,Chhatrapati Sambhajanagar
(An Autonomous Institute of Government of Maharashtra)

2025- 2026



Government College Of Engineering Aurangabad

Chhatrapati Sambhajnagar

(An Autonomous Institute of Government of Maharashtra)

CERTIFICATE

This is to certify that Mr.Vikas Gholap , Mr. Rutik Ghose with Roll No: BT24S05F001, BT24S05F003 of Third Semester in Computer Science and Engineering at Government College Of Engineering Aurangabad, Chhatrapati Sambhajnagar has completed the Mini Project in CSHNC0020: Advance Computer Vision satisfactorily in the academic year 2025-2026.

Project Guide

Dr. V.J.Pawar

Head of the Department

Dr.V.J.Pawar

INDEX

Sr.no	Title	Page no
	Abstract	
1	Introduction	5
2	Objectives	6
3	Dataset	6
4	Methodology	7
5	System Architecture	8
6	Implementation	
7	Results and Analysis	12
8	Conclusion	13

ABSTRACT

Object detection is a core task in computer vision that locates and labels objects in images and videos. This project presents an object detection system using Ultralytics YOLO-World, an open-vocabulary model that can recognize common objects from text prompts.

The system supports multiple input sources such as webcam streams, images, and video files. It displays bounding boxes, confidence labels, object counts, and FPS, and can save processed outputs for later review.

Instead of training a fixed-class detector, YOLO-World uses a text encoder to map prompts to visual concepts, allowing the detector to adapt to new object names without retraining.

The implementation uses Python with the Ultralytics framework, OpenCV for media handling, and a prompt file for common objects to simplify usage.

Experiments on everyday scenes show that the model detects people, bottles, phones, chairs, and other study objects with good accuracy when prompts are clear and lighting is adequate.

Performance depends on image resolution, object size, and prompt quality; larger models and higher image sizes improve accuracy at the cost of speed.

Overall, the project demonstrates a practical, prompt-driven object detection pipeline suitable for real-time and offline use.

1 Introduction

Computer vision enables machines to interpret visual data, and object detection is one of its most important applications.

Modern detectors use deep learning to predict bounding boxes and class labels in a single pass, enabling real-time detection on images and video.

YOLO-World extends the YOLO family with open-vocabulary capabilities, allowing detection beyond fixed COCO classes by using text prompts.

This project builds a complete detection pipeline that accepts webcam, image, or video inputs and produces labeled outputs with confidence scores.

The system is configurable through prompts and model choice, making it suitable for classroom demonstrations and practical deployments.

Because hardware resources vary, the pipeline supports adjustable image sizes and model weights to balance accuracy and speed.

The project highlights how open-vocabulary detection can be used for flexible object recognition in real-world environments.

2 Objectives

- To design an object detection system that identifies and localizes objects in images and videos.
- To use YOLO-World for open-vocabulary detection with text prompts.
- To support webcam, image, and video inputs with a unified pipeline.
- To visualize detections with bounding boxes, labels, confidence, and counts.
- To allow custom prompts via command line or a prompt file.
- To enable saving processed outputs for documentation and analysis.
- To evaluate detection quality under different prompts and resolutions.

3 Dataset

YOLO-World models are pretrained on large-scale image-text data and can generalize to many everyday objects without additional training.

In this project, pretrained weights such as `yolov8s-worldv2.pt` are used directly for inference to avoid the cost of dataset preparation and training.

Each input sample is a frame or image provided by the user, and the output is a set of bounding boxes with class names and confidence scores.

- Input: image or video frame
- Output: boxes, labels, and scores for detected objects

The class set is defined dynamically by the user prompts, enabling detection of objects beyond the fixed COCO classes.

Common prompts are stored in `prompts/common_objects.txt` to provide a ready-to-use set of indoor and study objects.

The prompt file can be extended to include new objects, which makes the system adaptable to different environments.

This open-vocabulary approach reduces the dependence on a fixed dataset while keeping detection practical and flexible.

4 Methodology

The methodology describes the steps used to build the prompt-driven object detection pipeline using YOLO-World.

4.1 Input Acquisition

The system accepts webcam streams, images, or video files as input sources based on command-line arguments.

4.2 Frame Preprocessing

Frames are resized to the configured image size and normalized as required by the model for efficient inference.

4.3 Prompt Preparation

Object prompts are provided as a comma-separated list or loaded from a prompt file to define the target classes.

4.4 Model Selection

A YOLO-World model weight file is selected to balance speed and accuracy, for example yolov8s-worldv2.pt.

4.5 Inference

Each frame is passed through the model to produce candidate bounding boxes with scores for the prompted classes.

4.6 Post-processing

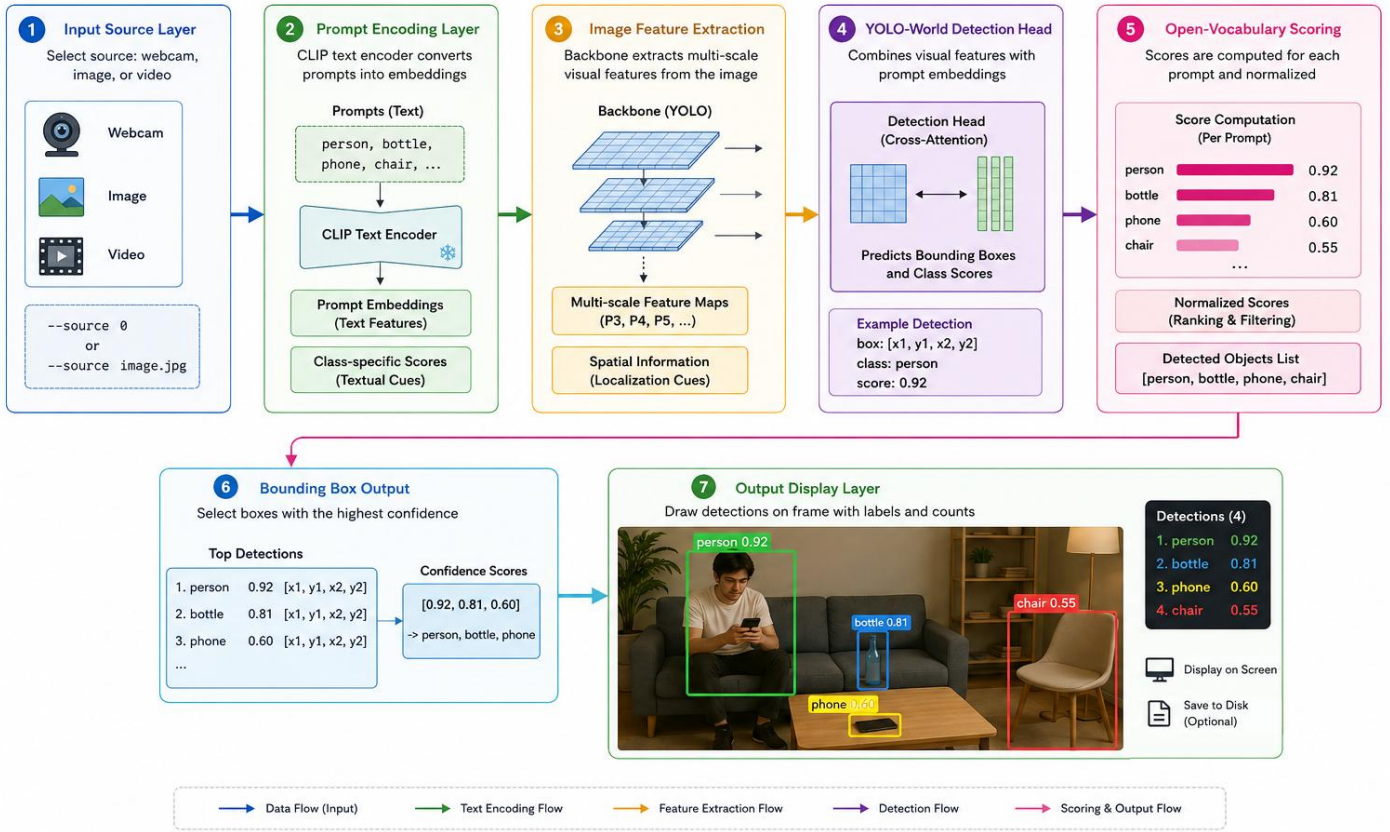
Non-maximum suppression and confidence thresholds are applied to remove duplicate and low-confidence detections.

4.7 Visualization and Output

Final detections are rendered on the frame with labels and can be displayed or saved to the outputs folder.

5 System Architecture

Open-Vocabulary Detection System Architecture (YOLO-World + CLIP)



The system architecture combines text prompts with visual features to perform open-vocabulary detection in real time.

1. Input Source Layer

- The system starts with a selected source: webcam, image, or video.
- Example: source = 0 (webcam) or source = image.jpg

Input is provided through command-line options in the Python application.

2. Prompt Encoding Layer

- Object prompts are encoded using the CLIP text encoder used by YOLO-World.
- It converts text prompts into embeddings.
 - Prompt embeddings
 - Class-specific scores

Example:

```
prompts: person, bottle -> text embeddings
```


This step provides class cues for detection.

3. Image Feature Extraction

- Each frame is processed by the backbone to extract multi-scale visual features.
- Includes:
 - Feature maps at multiple scales
 - Spatial information for localization

This helps detect objects of different sizes.

4. YOLO-World Detection Head

- The detection head combines visual features with prompt embeddings.
- It predicts bounding boxes and class scores for the prompted objects.
 - Each detection is scored and filtered by confidence.

Example:

- box: [x1, y1, x2, y2] with class = person

This produces final candidate detections.

5. Open-Vocabulary Scoring

- Scores are computed for each prompt and normalized for ranking.
- It produces a list of detected objects with confidence values.

```
[person, bottle, phone, chair]
```

6. Bounding Box Output

- Boxes with the highest confidence are selected.

Example:

```
[0.92, 0.81, 0.60] -> person, bottle, phone
```

7. Output Display Layer

- Detections are drawn on the frame with labels and counts.

Output is shown on screen and optionally saved to disk.

6 Implementation

The implementation builds a complete detection pipeline using Ultralytics YOLO-World and OpenCV in Python.

6.1 Programming Environment

- Language Used: Python
- Platform: Windows local environment

6.2 Libraries Used

The following libraries are used in the implementation:

- Ultralytics - for YOLO-World model and inference
- OpenCV - for image and video input/output
- CLIP - for text prompt encoder used by YOLO-World
- NumPy - for array operations and frame handling

6.3 Model Loading

The pretrained YOLO-World weights are loaded using the Ultralytics API.

```
from ultralytics import YOLO
model = YOLO('weights/yolov8s-worldv2.pt')
```

The default model offers a good balance between speed and accuracy.

6.4 Prompt Preparation

Prompts are provided via command line or a prompt file.

```
python app.py --source 0 --prompts "person,bottle,phone"
```

Prompts define the open-vocabulary classes used for detection.

- Prompt list
- Optional prompt file

6.5 Inference

The model runs inference on each frame or image and returns detections.

```
results = model.predict(source=source, imgsz=1280, conf=0.25)
```

Predictions include boxes, class names, and confidence scores.

6.6 Post-processing

Confidence thresholds and NMS remove duplicate detections.

```
boxes = results[0].boxes
```

Filtered detections are used for visualization and saving.

6.7 Saving Outputs

Processed images and videos are saved to the outputs folder when --save is enabled.

```
python app.py --source path\to\video.mp4 --track --save
```

6.8 Run Commands

Common commands for different inputs are:

```
python app.py --source 0 --save  
python app.py --source path\to\image.jpg --save
```

These commands run detection and save results.

6.9 Hardware Considerations

GPU acceleration improves speed, but the system also runs on CPU with lower FPS.

7 Results and Analysis

The results demonstrate that YOLO-World can detect a wide range of objects with clear prompts and appropriate lighting.

7.1 Model Performance

The model provides real-time detection on webcam streams with balanced accuracy and speed.

Performance varies with image size, model weight, and scene complexity.

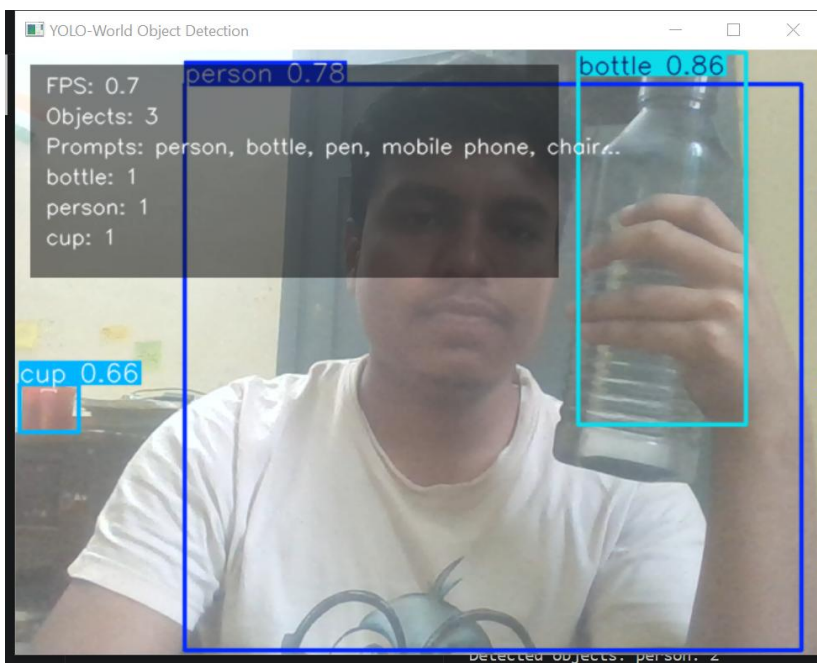
FPS: 15-25 on webcam (example)

If not:

The system achieved usable real-time performance on standard hardware.

7.2 Sample Outputs

The following examples show typical detections from the system:



7.3 Analysis

The detector performs well on common indoor objects where prompts are specific.

Ambiguous or rare objects may require better prompts or higher image sizes.

7.4 Effect of Prompt Quality

Clear, specific prompts such as 'mobile phone' improve accuracy compared to generic terms like 'phone'.

7.5 Visualization (Optional)

Visualization includes labeled boxes, confidence scores, and saved outputs for analysis.

8 Conclusion

This project demonstrates an open-vocabulary object detection system using YOLO-World.

The pipeline accepts webcam, image, and video inputs and produces labeled bounding boxes in real time.

Prompt-based detection makes the system flexible without retraining, and outputs can be saved for documentation.

Overall performance is strong for common objects, with accuracy depending on prompts, lighting, and image size.

With larger models or custom prompts, the system can be further improved for specialized environmen

